

PowerShell Commands Cheat Sheet

Essential cmdlets with clear syntax, real examples, and plain-English explanations.

CONTENTS	
01	Getting Help & Discovery
02	Variables & Data Types
03	File System Navigation
04	Working With the Pipeline
05	Processes & Services
06	Modules & Packages
07	Networking
08	Remoting & Sessions
09	Data Export & Formatting
10	Operators & Conditionals
11	Functions & Scripts

Tip: cmdlet names follow a Verb-Noun pattern (e.g. Get-Process). Most have short aliases shown in parentheses. Run any cmdlet with -? or Get-Help <name> -Examples for more detail.

01 Getting Help & Discovery

Get-Help

Displays help information about any cmdlet, function, or concept.

EXAMPLE

```
Get-Help Get-Process -Full
```

EXPLANATION

Shows the complete help page for Get-Process, including syntax, parameters, and examples. Use -Online to open the web version, or -Examples to see only usage samples.

Get-Command

Lists all available cmdlets, functions, and aliases in the current session.

EXAMPLE

```
Get-Command -Verb Get -Noun *service*
```

EXPLANATION

Finds every command whose verb is 'Get' and whose noun contains 'service' -- useful when you know roughly what you want but not the exact cmdlet name.

Get-Member

Shows the properties and methods of an object piped into it.

EXAMPLE

```
Get-Process | Get-Member
```

EXPLANATION

Reveals every property (Id, CPU, ProcessName, etc.) and method available on process objects -- helpful when scripting against specific object data.

Update-Help

Downloads and installs the newest help files for installed modules.

EXAMPLE

```
Update-Help -Force
```

EXPLANATION

Refreshes local help content from the internet; -Force re-downloads even if help was updated recently. Requires an elevated (Administrator) session.

02 Variables & Data Types

Setting a Variable

Stores a value in memory for later use; no explicit type declaration required.

EXAMPLE

```
$name = "Alex"
$age = 30
```

EXPLANATION

PowerShell infers the data type automatically (string and integer here). Variable names always begin with a dollar sign (\$).

\$null / Boolean Checks

Represents 'nothing', or tests whether a variable has no value.

EXAMPLE

```
if ($result -eq $null) { "Empty" }
```

EXPLANATION

\$null is a special automatic variable. Comparing with -eq \$null is safer than treating a variable as a boolean directly, especially with arrays.

Arrays

Holds an ordered collection of items in a single variable.

EXAMPLE

```
$fruits = @("Apple", "Banana", "Cherry")
$fruits[0]
```

EXPLANATION

@() defines an array explicitly. Index [0] retrieves the first element, since PowerShell arrays are zero-indexed.

Hash Tables

Stores key-value pairs, similar to a dictionary.

EXAMPLE

```
$user = @{ Name = "Sam"; Role = "Admin" }  
$user.Name
```

EXPLANATION

Hash tables suit structured data. Dot notation (\$user.Name) or bracket notation (\$user["Name"]) both retrieve a value for a given key.

\$env: Variables

Accesses operating system environment variables.

EXAMPLE

```
$env:USERNAME
```

EXPLANATION

The \$env: scope exposes system environment variables -- such as current username, computer name, or PATH -- without external tools.

03 File System Navigation

Get-ChildItem

Lists files and folders in a directory (alias: dir, ls).

EXAMPLE

```
Get-ChildItem -Path C:\Projects -Recurse -Filter *.ps1
```

EXPLANATION

Recursively searches C:\Projects and all subfolders for files ending in .ps1. -Filter narrows results early, which is faster than filtering afterward.

Set-Location

Changes the current working directory (alias: cd).

EXAMPLE

```
Set-Location -Path "C:\Users\Alex\Documents"
```

EXPLANATION

Moves the shell's context into the specified folder so relative paths used afterward resolve from there.

New-Item

Creates a new file, folder, or registry key.

EXAMPLE

```
New-Item -Path .\Logs -ItemType Directory
```

EXPLANATION

Creates a folder named 'Logs' in the current directory. Use -ItemType File instead to create an empty file.

Copy-Item

Copies files or folders from one location to another.

EXAMPLE

```
Copy-Item -Path .\report.docx -Destination D:\Backup\ -Force
```

EXPLANATION

Copies report.docx to the Backup drive. -Force overwrites an existing file at the destination without prompting.

Move-Item

Moves a file or folder to a new location (also used to rename items).

EXAMPLE

```
Move-Item -Path .\old.txt -Destination .\archive\old.txt
```

EXPLANATION

Relocates old.txt into the archive subfolder. Supplying a different file name at the destination effectively renames it.

Remove-Item

Deletes files, folders, or registry entries.

EXAMPLE

```
Remove-Item -Path .\temp\* -Recurse -Confirm:$false
```

EXPLANATION

Deletes everything inside the temp folder, including subfolders (-Recurse), without asking for confirmation on each item.

Test-Path

Checks whether a specified file, folder, or path exists.

EXAMPLE

```
if (Test-Path "C:\Data\config.json") { "Found" }
```

EXPLANATION

Returns \$true or \$false -- commonly used to guard scripts against errors before reading or writing a file.

Get-Content

Reads the contents of a file line by line (alias: cat, gc).

EXAMPLE

```
Get-Content -Path .\log.txt -Tail 20
```

EXPLANATION

Displays only the last 20 lines of log.txt -- useful for quickly checking recent entries in a large log file.

Set-Content / Add-Content

Writes or appends text to a file.

EXAMPLE

```
"Deployment finished" | Add-Content -Path .\log.txt
```

EXPLANATION

Appends the given text as a new line to log.txt. Set-Content instead overwrites the file's existing contents.

04 Working With the Pipeline

Where-Object

Filters objects in the pipeline based on a condition (alias: ?, where).

EXAMPLE

```
Get-Process | Where-Object { $_.CPU -gt 100 }
```

EXPLANATION

Passes only process objects whose CPU property exceeds 100 down the pipeline. \$_ refers to the current object being evaluated.

Select-Object

Chooses specific properties of an object, or limits result count.

EXAMPLE

```
Get-Process | Select-Object -Property Name, CPU -First 5
```

EXPLANATION

Returns only the Name and CPU columns for the first five processes, trimming both columns and row count from the output.

Sort-Object

Sorts pipeline objects by one or more properties.

EXAMPLE

```
Get-Process | Sort-Object -Property CPU -Descending
```

EXPLANATION

Orders the process list from highest to lowest CPU usage, making resource-heavy processes easy to spot at the top.

ForEach-Object

Runs a script block once for every object in the pipeline (alias: %).

EXAMPLE

```
1..5 | ForEach-Object { $_ * 2 }
```

EXPLANATION

Doubles each number from 1 to 5, producing 2, 4, 6, 8, 10. `$_` represents the current item in each iteration.

Measure-Object

Calculates statistics (count, sum, average, min, max) on pipeline data.

EXAMPLE

```
Get-ChildItem *.log | Measure-Object -Property Length -Sum
```

EXPLANATION

Adds up the Length (size in bytes) of every .log file in the current folder, giving a quick total disk usage figure.

Group-Object

Groups pipeline objects that share a common property value.

EXAMPLE

```
Get-Process | Group-Object -Property Company
```

EXPLANATION

Clusters running processes by the company that published them -- handy for auditing what software is active.

05 Processes & Services

Get-Process

Lists currently running processes on the system (alias: ps, gps).

EXAMPLE

```
Get-Process -Name "chrome"
```

EXPLANATION

Displays details -- CPU, memory, ID -- for every running process whose name matches 'chrome'.

Stop-Process

Terminates a running process.

EXAMPLE

```
Stop-Process -Name "notepad" -Force
```

EXPLANATION

Forcibly closes all running Notepad instances. Using -Id instead of -Name targets one specific process by its numeric ID.

Start-Process

Launches an application or executable (alias: saps, start).

EXAMPLE

```
Start-Process -FilePath "notepad.exe" -ArgumentList "C:\notes.txt"
```

EXPLANATION

Opens Notepad with notes.txt loaded. -Verb RunAs can be added to launch the process with elevated administrator rights.

Get-Service

Lists Windows services and their current status.

EXAMPLE

```
Get-Service | Where-Object { $_.Status -eq "Running" }
```

EXPLANATION

Filters the full service list down to only those services that are currently running, hiding stopped or disabled ones.

Restart-Service

Stops and restarts a Windows service in one step.

EXAMPLE

```
Restart-Service -Name "Spooler"
```

EXPLANATION

Restarts the Print Spooler service -- a common fix for print queue issues -- without needing separate stop and start commands.

06 Modules & Packages

Get-Module

Lists modules currently loaded, or all modules available to load.

EXAMPLE

```
Get-Module -ListAvailable
```

EXPLANATION

Shows every module installed on the system, whether or not it has been imported into the current session.

Import-Module

Loads a module's cmdlets into the current session.

EXAMPLE

```
Import-Module -Name ActiveDirectory
```

EXPLANATION

Makes Active Directory cmdlets (like Get-ADUser) available for use, assuming the module is installed.

Install-Module

Downloads and installs a module from a repository like PSGallery.

EXAMPLE

```
Install-Module -Name Az -Scope CurrentUser
```

EXPLANATION

Installs the Azure PowerShell module for the current user only, avoiding the need for administrator rights.

Set-ExecutionPolicy

Changes the script execution security setting.

EXAMPLE

```
Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned
```

EXPLANATION

Allows locally written scripts to run while still requiring downloaded scripts to be digitally signed -- balancing safety and usability.

07 Networking

Test-Connection

Sends ping requests to test network connectivity (modern replacement for ping).

EXAMPLE

```
Test-Connection -TargetName google.com -Count 4
```

EXPLANATION

Sends four ping packets to google.com and reports round-trip time and success rate for each.

Invoke-WebRequest

Sends an HTTP or HTTPS request and returns the response (alias: iwr, curl).

EXAMPLE

```
Invoke-WebRequest -Uri "https://api.example.com/data" -Method GET
```

EXPLANATION

Retrieves the content, headers, and status code from the given URL -- useful for quick API testing directly from the shell.

Invoke-RestMethod

Sends a web request and automatically parses JSON/XML responses.

EXAMPLE

```
Invoke-RestMethod -Uri "https://api.example.com/users" | Select-Object -First 3
```

EXPLANATION

Calls the API and converts the JSON response straight into PowerShell objects, then shows the first three records.

Get-NetIPAddress

Displays IP address configuration for network adapters.

EXAMPLE

```
Get-NetIPAddress -AddressFamily IPv4
```

EXPLANATION

Lists all IPv4 addresses currently assigned to network interfaces on the machine.

Resolve-DnsName

Performs a DNS lookup for a hostname (replacement for nslookup).

EXAMPLE

```
Resolve-DnsName -Name www.example.com
```

EXPLANATION

Returns the IP addresses and record type associated with the given domain name.

08 Remoting & Sessions

Enter-PSSession

Opens an interactive remote session to another computer.

EXAMPLE

```
Enter-PSSession -ComputerName Server01 -Credential (Get-Credential)
```

EXPLANATION

Connects interactively to Server01 using the supplied credentials; commands typed afterward run remotely until Exit-PSSession is used.

Invoke-Command

Runs a script block on one or more remote computers without an interactive session.

EXAMPLE

```
Invoke-Command -ComputerName Server01,Server02 -ScriptBlock { Get-Service }
```

EXPLANATION

Executes Get-Service on both servers simultaneously and returns combined results -- ideal for bulk administrative tasks.

New-PSSession

Creates a persistent, reusable remote session object.

EXAMPLE

```
$session = New-PSSession -ComputerName Server01
```

EXPLANATION

Stores an open connection to Server01 in \$session, which can be reused across multiple Invoke-Command calls to avoid reconnecting each time.

09 Data Export & Formatting

Export-Csv

Saves pipeline objects to a CSV file.

EXAMPLE

```
Get-Process | Export-Csv -Path .\processes.csv -NoTypeInfoation
```

EXPLANATION

Writes running process data to a CSV file. -NoTypeInfoation removes the extra type-metadata header line for cleaner output.

Import-Csv

Reads a CSV file and converts each row into an object.

EXAMPLE

```
$users = Import-Csv -Path .\users.csv
```

EXPLANATION

Loads users.csv into \$users, where each row becomes an object whose properties match the column headers.

ConvertTo-Json

Converts an object into JSON-formatted text.

EXAMPLE

```
$user | ConvertTo-Json -Depth 3
```

EXPLANATION

Serializes the \$user object into JSON, useful for sending data to a web API. -Depth controls how many nested levels are included.

Format-Table

Controls how output is displayed as a table in the console.

EXAMPLE

```
Get-Service | Format-Table -Property Name, Status -AutoSize
```

EXPLANATION

Displays results as a neatly aligned table showing only Name and Status columns, auto-sized to fit the content width.

10 Operators & Conditionals

Comparison Operators

Compares two values: -eq, -ne, -gt, -lt, -ge, -le.

EXAMPLE

```
if ($age -ge 18) { "Adult" } else { "Minor" }
```

EXPLANATION

PowerShell uses letter-based operators instead of symbols like == or >, since > and < are reserved for redirection.

Logical Operators

Combines multiple conditions: -and, -or, -not.

EXAMPLE

```
if ($isAdmin -and $isActive) { "Access granted" }
```

EXPLANATION

Grants access only when both conditions are true. -or requires just one condition to be true; -not inverts a condition.

Switch Statement

Evaluates a value against multiple possible matches.

EXAMPLE

```
switch ($day) {  
    "Mon" { "Start of week" }  
    "Fri" { "Almost weekend" }  
    default { "Midweek" }  
}
```

EXPLANATION

Runs the block matching \$day's value; if nothing matches, the default block runs -- similar to a case statement in other languages.

Wildcards

Matches patterns in strings or file names using * and ?.

EXAMPLE

```
Get-ChildItem -Filter "report_*.xlsx"
```

EXPLANATION

Matches any file beginning with 'report_' and ending in .xlsx, where * stands in for any number of characters.

11 Functions & Scripts

Defining a Function

Creates a reusable, named block of script logic.

EXAMPLE

```
function Get-Square {  
    param($Number)  
    return $Number * $Number  
}  
Get-Square -Number 5
```

EXPLANATION

Defines a function named Get-Square that accepts one parameter and returns its squared value; calling it with -Number 5 returns 25.

Try / Catch / Finally

Handles errors gracefully instead of letting a script crash.

EXAMPLE

```
try {  
    1/0  
} catch {  
    "Error: $($_.Exception.Message)"  
} finally {  
    "Done"  
}
```

EXPLANATION

Attempts the risky operation in try; if it throws, catch handles it and `$_Exception.Message` describes what went wrong; finally always runs.

Get-History

Lists commands run in the current session (alias: h, history).

EXAMPLE

```
Get-History | Select-Object -Last 10
```

EXPLANATION

Shows the ten most recently executed commands -- useful for recalling or re-running earlier work.

Comment-Based Help

Documents a script or function using structured comments.

EXAMPLE

```
<#  
.SYNOPSIS  
Squares a number.  
.PARAMETER Number  
The value to square.  
#>
```

EXPLANATION

Placed above a function, this block lets Get-Help display a proper synopsis and parameter description automatically.